

Contents

7	Array	1
7.1	Why do we need array?	1
7.2	One-dimensional array	2
7.2.1	Declaration and access of one-dimensional array . . .	2
7.2.2	Initialization of one-dimensional array	7
7.2.3	Searching and sorting	12
7.3	Two-dimensional array	17
7.3.1	Declaration and access of two-dimensional array . .	18
7.3.2	Initialization of two-dimensional array	18
7.3.3	Memory layout of two-dimensional array	18
7.4	Multi-dimensional array	18
7.5	Interesting points	18
7.6	Answers	19

Chapter 7

Array

Contents

7.1	Why do we need array?	1
7.2	One-dimensional array	2
7.2.1	Declaration and access of one-dimensional array	2
7.2.2	Initialization of one-dimensional array	7
7.2.3	Searching and sorting	12
7.3	Two-dimensional array	17
7.3.1	Declaration and access of two-dimensional array	18
7.3.2	Initialization of two-dimensional array	18
7.3.3	Memory layout of two-dimensional array	18
7.4	Multi-dimensional array	18
7.5	Interesting points	18
7.6	Answers	19

7.1 Why do we need array?

So far we have used primary data types in our programs (e.g. char, int, float, double). But sometimes primary data types alone are not enough to write C programs.

Say you want to write a C program which takes marks of students as input (until -1 is entered) and outputs deviation of marks of each student from average. In other words, program outputs - the difference of student's marks and average marks - for each student. For example if input marks

are 1, 2, 3, 4, 5, -1 then average would be 3. So output should be -2, -1, 0, 1, 2. Output is the deviation of each student's marks with the average marks.

How would you write C program to solve above problem? Can you write this program using only primary data types? No, because average can not be calculated until marks of all students have been entered by user, so program needs to remember marks of all students in order to calculate deviation from average. Now the problem is, how can program remember marks of all students simultaneously? If you declare variable named marks of type integer (**int marks;**), it can only remember (store) marks of one student at any given time.

In such situations array will come to your rescue. Using array you can store multiple (related) data items of same data type.

Array is a *sequenced* collection of *related* data items that share a common name.

Arrays can have one, two or more dimensions. Let us first look into one-dimensional array. Later we will dwell into two dimensional and multi dimensional array.

7.2 One-dimensional array

7.2.1 Declaration and access of one-dimensional array

When we declare any variable (e.g. *int total;*), it reserves memory - during execution of the program - for storing its value. And during execution of the program that memory can be accessed (read from or written to) using variable name.

Similarly, you can declare an array using following syntax.

data-type variable-name[size];

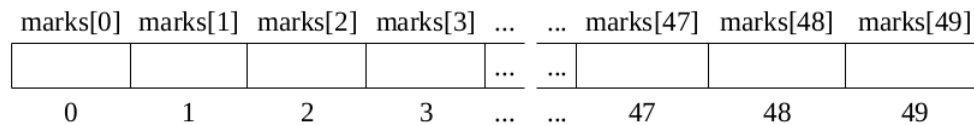
When we declare an array, it reserves memory - during execution of the program - for storing all the elements of that array. Every element of the array can be accessed (read from or written to) using array name and the index of the element to be accessed.

Here is an example of array declaration:

```
int marks[50];
```

Above statement declares array named *marks* of type *int*. It can store 50 integer values. Elements of this array would be indexed from 0 to 49.

Elements of an array would be stored contiguously in memory. Here is the visual representation of the array.



We can access first element of the array by name of the array - *marks* - and index of the first element. As index start from 0, index of the first element would be 0. Index value needs to be put within square brackets.

Following statement would store value 95 at index 0 (as first element) of array named *marks* declared above.

```
marks[0] = 95;
```

Following statement would assign - value stored at index 21 of array named *marks* - to variable *my_marks*.

```
my_marks = marks[21];
```

Assume that *index* is declared as integer variable. Then following statements would assign - value stored at index 21 of array named *marks* - to variable *my_marks*. It is equivalent to above statement.

```
index = 21;
my_marks = marks[index];
```

Following statements are also equivalent to above statements. So, index can be represented as integer constant, integer variable or integer expression (expression which evaluates to integer value).

```
index = 20;
my_marks = marks[index + 1];
```

Following statement would print value of stored at last element (at index 49) of array named *marks*.

```
printf("%d", marks[49]);
```

Following statement would store - value entered by user - at index 11 of array named *marks*.

```
scanf("%d", &marks[11]);
```

Following statements would compile silently without any error, but during execution they may lead to unpredictable behaviour as they are trying to access elements outside valid indices (0-49) for array named *marks* declared above.

```
marks[-1] = 99;
marks[50] = 33;
marks[77] = 66;
marks[-10] = 45;
```

Following statement is also invalid and would result in syntax error - **array subscript is not an integer.**

```
marks[21.5] = 99;
```

Here are few more examples of valid array declaration:

```
char surname[30];
float weights[10];
long distances[10];
```

Character array (a.k.a. string) will be discussed in depth in next chapter. So we will not talk about arrays of type *char* in this chapter.

Now that we know how to declare an array and access array elements, let us write a simple C program which takes 5 floating point numbers from user and prints them in reverse.

To print numbers in reverse, program will need to remember all the numbers that user have entered. To store these numbers, we can use array of type *float*. In the following code, first *for* loop is just scanning the numbers in the array; and second *for* loop is printing numbers stored in array in reverse.

```
1 #include<stdio.h>
2
3 #define NUMBERS_TO_SCAN 5
4
5 int main()
6 {
7     float numbers[NUMBERS_TO_SCAN];
8     int i;
9
10    for (i = 0; i < NUMBERS_TO_SCAN; i++)
11    {
12        scanf("%f", &numbers[i]);
13    }
14
15    for (i = NUMBERS_TO_SCAN - 1; i >= 0; i--)
16    {
```

```

17         printf("%.2f\n", numbers[i]);
18     }
19
20     return 0;
21 }

```

Above program just prints the array in reverse. It is not changing the actual content of the array. Can you write a program which actually reverses the array and then print the reversed array? Do it on your own for practice.

Let us try to write a C program for the problem with which we started the chapter. A C program which takes marks of students as input (until -1 is entered) and outputs deviation of marks of each student from average. In other words, program outputs - the difference of student's marks and average marks - for each student.

```

1  #include<stdio.h>
2
3  #define MAX_STUDENTS 100
4
5  int main()
6  {
7      int marks[MAX_STUDENTS], students = 0;
8      int i, sum = 0;
9      float average, deviation;
10
11     scanf("%d", &marks[0]);
12     while (-1 != marks[students])
13     {
14         sum += marks[students];
15         students++;
16         scanf("%d", &marks[students]);
17     }
18
19     if (0 == students)
20     {
21         printf("No marks entered\n");
22     }
23     else
24     {
25         average = (float) sum / students;

```

```

26         for (i = 0; i < students; i++)
27         {
28             deviation = marks[i] - average;
29             printf("%.2f\n", deviation);
30         }
31     }
32
33     return 0;
34 }

```

We have used array named *marks* to store marks of students entered by user. *while* loop have been used to take marks from user. While loop will terminate if user enters -1 value. It uses variable *students* to keep track of number of students whose marks have been entered by user. It correctly handles corner cases like - no marks are entered - or - if marks are entered for only one student. *for* loop has been used to calculate deviation from mean and to print the deviation. Please note names of variables and indentation. You should always indent your code properly and use meaningful names for variables used in the program. These habits make programs readable and maintainable by other programmers.

Now try to extend the above program to find standard deviation of marks entered by user. If you do not remember - from your maths class - what is standard deviation, then search engine can help you learn it. Please make sure that you write well indented code with meaningful names of variables.

NOTE: There are two points to note, first - 'I will keep emphasizing importance of naming convention and indentation' and second - 'I will keep emphasizing importance of naming convention and indentation' :)

Try to answer following questions before moving on to the next section. Correct answers are given at the end of the chapter. But those answers are for your reference, do not directly jump to answers without trying to answer the questions on your own.

Que 1:

Following code will generate compile time error. True/False?

```

int arr[5];
arr[5] = 3;

```

Que 2:

Is following code valid? If no, why? If yes, what would be the number

stored in variable *value* after execution of all these statements?

```
int arr[3], value = 1;
arr[value + 1] = value + 5;
value = arr[value + 1];
```

Que 3:

Is following code valid? If no, why? If yes, what would be the number stored in variable *value* after execution of all these statements?

```
float arr[3], value = 1.1;
arr[value + 1] = value + 5;
value = arr[value + 1];
```

7.2.2 Initialization of one-dimensional array

When we declare any variable, it can be initialized with a valid value at the time of declaration (e.g. *float number = 5.6;*). Here we initialized variable *number* with value 5.6.

Similarly, elements of the array can be initialized at the time of declaration. Following is the syntax for initializing elements of array at the time of declaration. Such initialization is also called **compile time initialization** as elements of the array would be initialized at compile time.

data-type variable-name[size] = {value 1, value 2, ..., value n};

value 1 would be stored at index 0, *value 2* would be stored at index 1 and so on. For example, following statement will create array *marks* of size 3 and will assign 5, 10, and 7 to *marks[0]*, *marks[1]*, and *marks[2]* resp.

```
int marks[3] = {5, 10, 7};
```

Here is the visualization of the array initialized by above statement.

marks[0]	marks[1]	marks[2]
5	10	7
0	1	2

If you initialize an array at the time of declaration then *size* is optional during array declaration. For example, following statement will create array *marks* of size 3 and will assign 5, 10, and 7 to *marks[0]*, *marks[1]*, and *marks[2]* resp. Here size of an array is decided by the number of elements initialized.

```
int marks[] = {5, 10, 7};
```


If you specify size as well as initialize array elements; and if initialization has less values than size specified then remaining elements will be initialized to zero. For example, following statement will create array *marks* of size 5. First three elements will be initialized with values 5, 10 and 7 resp. Remaining two elements will be initialized to 0.

```
int marks[5] = {5, 10, 7};
```

During programming we frequently need to initialize all elements of array to 0. Following is a good way to initialize all elements of array to 0.

```
int marks[5] = {0};
```

If initialization has more values than size specified then that's a constraint violation. Some compilers may just give warning. AVOID writing such code.

```
int marks[3] = {5, 10, 7, 2, 11};
```

If you do not initialize an array during declaration then all elements of an array will contain garbage values, until you assign some value to them. Following statement will create array *marks* of size 3, and all the elements will have some garbage value stored in them.

```
int marks[3];
```

Below is the summary of compile time initialization.

<code>int marks[3] = {5, 10, 7};</code>	Declares an array of size 3 and initialize its elements to values 5, 10 and 7 resp.
<code>int marks[] = {5, 10, 7};</code>	Declares an array of size 3 and initialize its elements to values 5, 10 and 7 resp.
<code>int marks[5] = {5, 10, 7};</code>	Declares and initializes array of size 5. First three elements are initialized to 5, 10, and 7 resp. Last two elements are initialized to 0.
<code>int marks[5] = {0};</code>	Declares an array of size 5 and initializes all elements to value 0.
<code>int marks[2] = {1, 2, 3};</code>	Constraint violation. Some compilers may just give warning. Should be avoided.
<code>int marks[3];</code>	Declares an array of size 3. All elements contain garbage value.

You can initialize array elements at run time and such initialization is called **run time initialization**. Following code segment is initializing each array element by - square of the index of an element. After declaration statement, all elements will have some garbage value. When *for* loop executes, it will start initializing elements of the array one by one (one element per iteration) - to the square of the index of an element. This is run time initialization.

```
int arr[100], i;
for(i = 0; i < 100; i++)
    arr[i] = i * i;
```

Values can also be assigned to array elements using read functions like *scanf*, during run time. Following is also an example of run time initialization.

```
int arr[100], i;
for(i = 0; i < 100; i++)
    scanf("%d", &arr[i]);
```

Let's now write a simple C program which calculates frequencies of marks. So it takes marks of students as an input and outputs number of students with 0 marks, number of students with 1 marks and so on.

```
1  #include<stdio.h>
2
3  #define MAX_STUDENTS 100
4  #define MAX_MARKS 36
5
6  int main()
7  {
8      int marks[MAX_STUDENTS], students = 0;
9      int frequencies[MAX_MARKS + 1] = { 0 }, i;
10     int input_marks;
11
12     while (1)
13     {
14         scanf("%d", &input_marks);
15         if (-1 == input_marks)
16             break;
17         else if (input_marks >= 0 &&
18                 input_marks <= MAX_MARKS)
19         {
20             marks[students] = input_marks;
21             students++;
22         }
23         else
24             printf("Marks out of range, "
25                    "please reenter\n");
26     }
27
28     for (i = 0; i < students; i++)
29         frequencies[marks[i]]++;
30
31     printf("MARKS \t STUDENTS\n");
32     for (i = 0; i < MAX_MARKS + 1; i++)
33         printf("  %d \t\t %d\n", i, frequencies[i]);
34
35     return 0;
36 }
```

In the above program, *while* loop has been used to take input from user. Inside *while* loop *scanf* statement has been used to take marks from user. After *scanf* statement it is checking if input marks is -1. If input marks is -1 its breaking the while loop and stops taking marks from user. If marks entered is not -1, then it checks if marks entered are within valid range. If marks are within valid range it adds marks to the *marks* array and increments the students count, and then it will ask user for next input (marks). If marks are not within valid range then it prints message for user to reenter the marks, and goes for the next iteration of the *while* loop.

First *for* loop in the above program is the core logic of the program and it calculates the frequencies of the marks. For all the elements of the *marks* array, it checks the value of the element (marks of a student) and then uses that value (marks of a student) as an index to access *frequencies* array. It increments an element of *frequencies* array - element at index represented by marks of a student. Please note that each index of *frequencies* array is representing one valid value of marks. Core logic is just two lines of code :). Second *for* loop is just printing the content of *frequencies* array.

Please note the *else if* condition within *while* loop in the above program. One statement has been broken up in two lines. It is totally valid and good programming practice. Whenever any line of code seems of be longer, you should break that statement in to multiple lines. Same is the case with *printf* statement within *else*. You can provide multiple strings as an argument to *printf* function, it will combine them all and print the string on the screen. Please note that there is no *comma* (,) between two strings in *printf* statement.

Let's test what you learned in this section.

Que 4:

Is following code valid? If no, why? If yes, what would be printed by it?

```
int arr[5] = {1, 10, 4};
printf("%d %d", arr[1], arr[3]);
```

Que 5:

Is following code valid? If no, why? If yes, what would be printed by it?

```
int arr[] = {1, 10, 4};
printf("%d %d", arr[0], arr[2]);
```

Que 6:

Is following code valid? If no, why? If yes, what would be printed by it?

```
int arr[3];
```

```
printf("%d %d", arr[0], arr[2]);
```

Que 7:

Is following code valid? If no, why? If yes, what would be printed by it?

```
int arr[3] = {0};  
printf("%d %d", arr[0], arr[2]);
```

Que 8:

Is following code valid? If no, why? If yes, what would be printed by it?

```
int arr[3] = {1, 4, 8, 1, 10};  
printf("%d %d", arr[0], arr[2]);
```

Que 9:

Is following code valid? If no, why? If yes, what would be printed by it?

```
int arr[10], i;  
for(i = 0; i < 5; i++)  
    arr[i] = i * i;  
for(i = 0; i < 6; i++)  
    printf("%d ", arr[i]);
```

Que 10:

Is following code valid? If no, why? If yes, what would be printed by it?

```
int arr[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 0};  
int sum = 0, i;  
for(i = 0; i < 10; i++)  
    if(i % 2 == 0)  
        sum += arr[i];  
printf("%d", sum);
```

Que 11:

Is following code valid? If no, why? If yes, what would be printed by it?

```
int arr[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 0};  
int sum = 0, i;  
for(i = 0; i < 10; i++)  
    if(arr[i] % 2 == 0)  
        sum += arr[i];  
printf("%d", sum);
```

7.2.3 Searching and sorting

One operation that you may frequently need to perform on an array is searching for a value. There are various algorithms for searching - sequen-

tial search (a.k.a. linear search), binary search, exponential search etc. We will only look into linear search.

Other operations like finding minimum number or finding maximum number from an array are also performed frequently on one dimensional array. We will look into program to find minimum number from one-dimensional array.

One more operation that you may frequently need to perform on an array is sorting the elements of the array. Sorting means arranging elements of array in ascending or descending order, based on some criteria. There are various algorithms for sorting - selection sort, bubble sort, merge sort, insertion sort, heap sort, quick sort etc. We will only look into selection sort.

Following program searches an array for first occurrence of a given number.

```
1  #include<stdio.h>
2
3  #define ARRAY_SIZE_MAX 1000
4
5  int main()
6  {
7      float arr[ARRAY_SIZE_MAX], element_to_be_searched;
8      int elements_in_array, i;
9
10     printf("Enter number of elements in array: ");
11     scanf("%d", &elements_in_array);
12
13     for (i = 0; i < elements_in_array; i++)
14     {
15         printf("Enter element %d: ", i + 1);
16         scanf("%f", &arr[i]);
17     }
18
19     printf("Enter element to be searched: ");
20     scanf("%f", &element_to_be_searched);
21
22     // Searching an array
23     for (i = 0; i < elements_in_array; i++)
24         if (arr[i] == element_to_be_searched)
25             break;
26
```

```

27     if (i == elements_in_array)
28         printf("Value %f not found in array\n",
29             element_to_be_searched);
30     else
31         printf("First occurrence of %f is at "
32             "index %d\n", element_to_be_searched,
33             i);
34
35     return 0;
36 }

```

This program first takes number of elements in array from user. Then first *for* loop is taking elements of array from user one by one (one element of array per iteration). Then it takes number to be searched from user. Second *for* loop in the program is searching for that number in array. Finally, it will check if value of variable *i* is equal to variable *elements_in_array*. If both of them are equal, it means all iteration of the second *for* loop were executed and *break* statement was not executed. Which means value is not present in the array. If value of variable *i* is not equal to variable *elements_in_array*, it means all iteration of the second *for* loop were not executed and loop was stopped by *break* statement. Which means value is present in the array at index represented by variable *i*.

Can you modify this program to print all indices (instead of only first) at which given number is present?

Following program finds smallest (minimum) number from one-dimensional array.

```

1  #include <stdio.h>
2
3  #define ARRAY_SIZE_MAX 1000
4
5  int main()
6  {
7      float arr[ARRAY_SIZE_MAX], minimum;
8      int elements_in_array, i;
9
10     printf("Enter number of elements in array: ");
11     scanf("%d", &elements_in_array);
12
13     if (0 == elements_in_array)
14     {

```

```

15         printf("No elements in the array\n");
16         return 0;
17     }
18     else
19     {
20         for (i = 0; i < elements_in_array; i++)
21         {
22             printf("Enter element %d: ", i + 1);
23             scanf("%f", &arr[i]);
24         }
25
26         // finding minimum number from an array
27         minimum = arr[0];
28         for (i = 0; i < elements_in_array; i++)
29             if (arr[i] < minimum)
30                 minimum = arr[i];
31
32         printf("Minimum number is %f\n", minimum);
33     }
34
35     return 0;
36 }

```

a

Now let us try to understand how to sort an array using **selection sort**. During selection sort array is logically divided into two portions. One portion has all sorted elements (left portion) and another portion has all unsorted elements (right portion). Algorithm starts with all elements in unsorted portion. Then algorithm checks through all unsorted elements and finds smallest element among unsorted elements. It swaps that smallest element of the unsorted portion with the first element of unsorted portion. So, now that smallest element becomes part of sorted array and unsorted array will now have one less element. It repeats this procedure until all elements of the array are sorted.

Let us try first try to understand how selection sort works with the help of an example. Consider following array of 5 elements. All the elements are in unsorted portion to start with.

5	4	1	3	2
---	---	---	---	---

During first pass program finds smallest element from unsorted portion

of the array. In this case, it is 1 at index 2. So it will be swapped with first element of unsorted portion of array. In this case, first element of unsorted portion is 5 at index 0. At the end of first pass, array would be in the state as shown below. Now sorted portion has one element (at index 0). Other elements (from index 1 to 4) are in unsorted portion. We will be showing sorted portion using underline below sorted elements.

<u>1</u>	4	5	3	2
----------	---	---	---	---

During second pass algorithm will again search for the smallest element in unsorted portion of the array (from index 1 to 4). Now smallest element in unsorted portion is 2 at index 4. It will be swapped with first element of the unsorted portion of the array - which is value 4 at index 1. At the end of the second pass, array would be in the state as shown below.

<u>1</u>	<u>2</u>	5	3	4
----------	----------	---	---	---

Similarly, at the end of third pass, array would be in the state as shown below.

<u>1</u>	<u>2</u>	<u>3</u>	5	4
----------	----------	----------	---	---

At the end of the fourth pass, array would be in the state as shown below.

<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>	5
----------	----------	----------	----------	---

Now, if all elements are sorted - except last element - then it means last element is greater than all other elements and hence it is sorted. Because greatest element of all the elements, would anyways be last element in sorted array. Here, to sort 5 elements we used 4 passes (over unsorted portion). Similarly, to sort n elements we will need $n - 1$ passes.

This algorithm is called **selection sort** because during each pass, it **selects** the smallest element from unsorted portion of the array and swaps it with the first element of unsorted portion of the array.

Here we showed how to sort an array in ascending order. Selection sort can also be used to sort an array in descending order. Only change in the algorithm would be that - instead of selecting smallest element from unsorted portion, you need to select greatest element from unsorted portion, and swap it with the first element of unsorted portion of the array.

Following program would sort an array in ascending order using selection

sort algorithm. First it takes number of elements in the array from user. Then first *for* loop is taking elements of array from user one by one (one element of array per iteration).

Then it uses two nested *for* loops for sorting the array. One iteration of the outer *for* loop (with control variable *i*), represents one pass of selection sort algorithm explained above. Inner *for* loop (with control variable *j*) is finding the index of the smallest element in unsorted portion of the array for current pass. Three statements outside inner *for* loop are swapping smallest element in unsorted portion of the array with first element of unsorted portion of the array. Value of variable *i* is dividing the sorted and unsorted portion. All elements on left of index represented by value of variable *i* are sorted and all elements on right of index represented by value of variable *i* are unsorted (element at index represented by value of variable *i* is also unsorted).

Last *for* loop is just printing the element of sorted array.

```
1 #include<stdio.h>
2
3 #define ARRAY_SIZE_MAX 1000
4
5 int main()
6 {
7     float arr[ARRAY_SIZE_MAX];
8     int elements_in_array, i, j, temp, min_index;
9
10    printf("Enter number of elements in array: ");
11    scanf("%d", &elements_in_array);
12
13    printf("Enter elements of the array\n");
14    for (i = 0; i < elements_in_array; i++)
15        scanf("%f", &arr[i]);
16
17    // Sorting an array
18    for (i = 0; i < elements_in_array - 1; i++)
19    {
20        min_index = i;
21        for (j = i + 1; j < elements_in_array; j++)
22        {
23            if (arr[j] < arr[min_index])
24                min_index = j;
25        }
```

```

26         temp = arr[i];
27         arr[i] = arr[min_index];
28         arr[min_index] = temp;
29     }
30
31     for (i = 0; i < elements_in_array; i++)
32         printf("%f ", arr[i]);
33
34     return 0;
35 }

```

Now that you know how to use selection sort algorithm to sort elements of an array, can you modify this program to sort elements of an array in descending order (instead of ascending order)?

7.3 Two-dimensional array

In the previous section we looked into one-dimensional array. Statement *int marks[50];* would declare an array named *marks* of type *int* and will reserve *contiguous* memory to store 50 integer values indexed from 0 to 49. Here, *marks* is an array of integers. Is there a way to create an **array of arrays** in C language?

An Array of one-dimensional arrays is known as two-dimensional array. In other words two-dimensional array is nothing but an array whose elements are arrays themselves. You can declare two-dimensional array using following syntax.

data-type variable-name[rows][columns];

Here is an example of declaration of two-dimensional array:

```
int _2d_arr[2][3];
```

Here, *_2d_arr* is an array of two elements. Each element of *_2d_arr* is an array of three integers. So in total it can store six integers. It can also be conceptualized as two rows of three integers each. Following is logical representation of *_2d_arr*.

	0	1	2
0	<code>_2d_arr[0][0]</code>	<code>_2d_arr[0][1]</code>	<code>_2d_arr[0][2]</code>
1	<code>_2d_arr[1][0]</code>	<code>_2d_arr[1][1]</code>	<code>_2d_arr[1][2]</code>

Like one-dimensional array, all elements of two-dimensional array are contiguously stored in memory. First element of `_2d_arr` (first row - `_2d_arr[0]`) would be followed by second element of `_2d_arr` (second row - `_2d_arr[1]`). Following diagram shows how elements of `_2d_arr` would be stored in memory.

To access integer element in above array, we need two indices. First index is for row (index of element of `_2d_arr` - which would be one-dimensional array in itself); second index is for column (index of integer within one-dimensional array). `_2d_arr[1][0]` represents first integer in second row. `_2d_arr[0][2]` represents third integer in first row.

<code>_2d_arr[0][0]</code>	<code>_2d_arr[0][1]</code>	<code>_2d_arr[0][2]</code>	<code>_2d_arr[1][0]</code>	<code>_2d_arr[1][1]</code>	<code>_2d_arr[1][2]</code>
First element (row) of <code>_2d_arr</code> (<code>_2d_arr[0]</code>)			Second element (row) of <code>_2d_arr</code> (<code>_2d_arr[1]</code>)		

Following is the logical representation of little larger array - `arr[20][50]`.

	0	1	2	...	...	47	48	49
0	<code>arr[0][0]</code>	<code>arr[0][1]</code>	<code>arr[0][2]</code>	...	...	<code>arr[0][47]</code>	<code>arr[0][48]</code>	<code>arr[0][49]</code>
1	<code>arr[1][0]</code>	<code>arr[1][1]</code>	<code>arr[1][2]</code>	...	...	<code>arr[1][47]</code>	<code>arr[1][48]</code>	<code>arr[1][49]</code>
2	<code>arr[2][0]</code>	<code>arr[2][1]</code>	<code>arr[2][2]</code>	...	...	<code>arr[2][47]</code>	<code>arr[2][48]</code>	<code>arr[2][49]</code>
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
17	<code>arr[17][0]</code>	<code>arr[17][1]</code>	<code>arr[17][2]</code>	...	...	<code>arr[17][47]</code>	<code>arr[17][48]</code>	<code>arr[17][49]</code>
18	<code>arr[18][0]</code>	<code>arr[18][1]</code>	<code>arr[18][2]</code>	...	...	<code>arr[18][47]</code>	<code>arr[18][48]</code>	<code>arr[18][49]</code>
19	<code>arr[19][0]</code>	<code>arr[19][1]</code>	<code>arr[19][2]</code>	...	...	<code>arr[19][47]</code>	<code>arr[19][48]</code>	<code>arr[19][49]</code>

7.3.1 Declaration and access of two-dimensional array

7.3.2 Initialization of two-dimensional array

7.3.3 Memory layout of two-dimensional array

7.4 Multi-dimensional array

7.5 Interesting points

Following statement is **VALID** and would store value 95 at index 0 of array named *marks*. So, either array name or index can be put within square brackets. Though this syntax is rarely used, you should know it. Because if you are reading code written by someone else and if s/he has chosen following syntax for some reason, then you should know that it is a valid syntax.

```
0[marks] = 95;
```

Following statements are **INVALID** and would result in syntax error. Square brackets can only be used around whatever (array name or index) you mention second.

```
[marks]0 = 95;  
[0]marks = 95;
```

7.6 Answers

Ans 1:

False.

Size of array *arr* is 5. So valid range of indices is 0-4. But code is trying to access element at index 5. Though it is trying to access element outside valid range of indices, it will not generate compile time error. It will compile silently without error, but it will exhibit unpredictable behavior during run time. So you should avoid code like this even if its not generating compile time error.

Ans 2:

Code is valid.

Number stored in variable *value* after execution of all these statements would be 6.

Ans 3:

Code is NOT valid.

Because variable *value* is of type *float*. In second statement we are trying to store number 6.1, at index 2.1. Index should always be integer and it can not be floating point number. Hence second statement would generate compile time error. Same is the problem with third statement.

Ans 4:

It is valid code and it will print

10 0

As array index starts from 0 in C language, *arr[0]* will have value 1 and *arr[1]* will have value 10, and *arr[2]* will have value 4. During array declaration if array is partially initialized then remaining elements are initialized to 0. Hence, remaining two elements *arr[3]* and *arr[4]* will be initialized to 0.

Ans 5:

It is valid code and it will print

1 4

Size of the array is not provided during declaration, so size of the array will be equal to number of elements initialized.

Ans 6:

It is valid code.

It will print two garbage values separated by space.

When array is not initialized all elements of the array contain some garbage value.

Ans 7:

It is valid code and it will print

0 0

Only first element of the array is initialized with value 0. During array declaration if array is partially initialized then remaining elements are initialized to 0.

Ans 8:

It is not valid code.

Because it violates the constraint that array initialization can not have more values than size of the array. A compiler may just show warning for this and still compile the code, but it's not a valid code and should be avoided.

Ans 9:

It is valid code and it will print

0 1 4 9 16 [garbage]

Only first five elements of the array have been initialized at run time. So rest of the array elements will contain garbage.

Ans 10:

It is valid code and it will print

25

Code is summing up the elements of array which are at even indices ($arr[0] + arr[2] + arr[4] + arr[6] + arr[8]$).

Ans 11:

It is valid code and it will print

20

Code is summing up the elements of array which have even values ($arr[1] + arr[3] + arr[5] + arr[7] + arr[9]$).